Cheatsheet · 1 chapters on one page

Manual API Testing cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

38 key ideas | 0 formulas | 37 tables | 24 tips and traps

STEP 1 Manual API testing course

1.1 Manual API Testing: Basics to Expert

Every app you use talks to a server through APIs. This course teaches you to test those APIs by hand, the way a tester explores them before anyone writes automation. You start with a single HTTP request and finish with contract testing, security testing and the newest topics such as testing AI APIs. Every part has an animation or a tool you can use, and the live playground lets you send real requests to a mock shop API with bugs hidden inside it.

KEY IDEAS

- **What an API is and why we test it** An API (Application Programming Interface) is a set of rules that lets one program ask another program for data or for an action.
- **What a web service is** A web service is an API that one program calls over a network using web standards such as HTTP, XML or JSON. Two systems use it to exchange data no matter which language or operating system each one runs on.
- **REST in depth** REST was defined by Roy Fielding in his doctoral dissertation in 2000. He was also one of the authors of the HTTP specification. An API that follows the REST constraints is called RESTful .
- **Kinds of APIs and where they sit in a system**
- **How HTTP carries a request** Most APIs you will test use HTTP (Hypertext Transfer Protocol), and almost always HTTPS , which is HTTP inside an encrypted TLS connection. HTTP is a request and response protocol. The client sends one request, the server sends back one response.
- **HTTP methods, safety and idempotency** The method tells the server what you want to do with the resource. Two properties matter a lot for testing. A safe method only reads and never changes data. An idempotent method gives the same server state whether you send it once or ten times.
- **Status codes** The three digit status code is the first thing you check. The first digit gives the family.
- **Headers, cookies and content negotiation** Headers carry metadata about the request or the response. Header names are case insensitive, so content-type and Content-Type are the same header.
- **JSON, XML and other payloads** JSON (JavaScript Object Notation) is the default format for modern APIs. It has six value types: string, number, boolean, null, object and array. Keys must be in double quotes, and trailing commas are not allowed.
- **REST, SOAP, GraphQL, gRPC, WebSocket and webhooks** REST is covered in depth in Part 3 . The quick reminders below are the naming habits you will check most often.
- **Tools: Postman, curl, Bruno and friends**
- **Seeing real traffic: DevTools, HAR files and proxies** Documentation tells you what an API should receive. Real traffic tells you what the app actually sends. Many of the best API bugs are found by watching the calls a web page or a mobile app makes while you click around, then replaying them with small changes.
- **Reading API docs and OpenAPI** Your test oracle is the contract. For REST APIs this is usually an OpenAPI document (formerly called Swagger) in YAML or JSON. It lists every path, method, parameter, request body, response, status code and security scheme.
- **Authentication and authorization** Authentication answers who you are. Authorization answers what you are allowed to do. A missing or bad identity should give 401. A known identity without permission should give 403 (or 404 if the API hides that the resource exists).

- **Types of API testing and when to use each** "API testing" is not one activity. It is a family of test types, and each one answers a different question. A hospital gives a good picture. A quick pulse check at the door is a smoke test. A full blood report is functional testing.
- **Designing API test cases** The test design techniques from manual testing all apply to APIs. The difference is that you control every byte of the request, so you can reach cases the UI never allows.
- **What to check in every response** A 200 status alone proves very little. A response can be 200 and still wrong. Check these layers every time.
- **Variables, chaining and end to end flows** Real flows need data from earlier responses. You log in, save the token, create an order, save its id, then read, update and delete that same order. This is called request chaining .
- **Pagination, filtering and sorting** No API returns a million rows at once. Lists come in pages. There are three common styles.
- **Rate limits, caching and idempotency keys** APIs limit how many requests a client can make, to protect the service and to stop brute force attacks. When you exceed the limit you should get 429 Too Many Requests with a Retry-After header.
- **Error handling and Problem Details** Good errors tell the client what went wrong and how to fix it, without leaking internals. The standard format is Problem Details for HTTP APIs , RFC 9457 (2023), which replaced RFC 7807. It uses the media type application/problem+json .
- **File uploads and downloads** Profile photos, resumes on a job site, invoices in a banking app and medical reports in a health app all travel through APIs. Files bring their own bugs: huge sizes, wrong types, broken names and security holes.
- **Dates, money, text and language edge cases** Some of the most famous software bugs had nothing to do with clever logic. They were about a clock, a currency or a name. Airline systems have shown flights landing before they took off because of time zones.
- **Concurrency, race conditions and double clicks** Picture the last train ticket on a festival weekend. Two people in two cities press "Book" in the same second.
- **Checking side effects: database, queues, emails and logs** A response of 201 Created tells you what the API claims happened. It does not prove it.
- **HTTPS, TLS certificates, CORS and HTTP versions** Before your request reaches any business logic, it passes through a few lower layers: encryption, certificates and browser safety rules. When these break, the whole app breaks, often with confusing errors.
- **API security testing: the OWASP API Top 10** The OWASP API Security Top 10 (2023 edition) is the standard list of the most common API risks. Manual testers find many of these with nothing more than Postman and curiosity. Only test systems you are allowed to test.
- **Performance testing for APIs** Even manual testers should note response times and payload sizes, and know the vocabulary of load testing.
- **Async APIs: webhooks, polling and queues** Some work takes too long for one request, such as generating a report or processing a video. The API answers 202 Accepted with a job id, and the client either polls a status endpoint or waits for a webhook.
- **Real world scenario walkthroughs** Here is how API testing looks on three real kinds of product. Pick a scenario to step through the calls, then read the test ideas a senior tester would bring to it.
- **Contract testing and versioning** When ten teams depend on one API, a small change such as renaming a field can break apps that nobody on the provider team knows about. Contract testing catches this before release.
- **Mocks, service virtualization and test data** You often need to test an API whose dependencies are not ready, are expensive (a real payment provider), or cannot be made to fail on demand. A mock server pretends to be that dependency.
- **Security tools and fuzzing: ZAP, Burp Suite and Schemathesis** Manual thinking finds the clever bugs. Tools find the many small ones a person would never have time to try. Fuzzing means sending large numbers of unexpected, random or malformed inputs and watching for crashes, 500 errors, slow responses or data leaks.
- **Testing GraphQL, gRPC, WebSocket and SSE** gRPC has its own status codes such as OK , INVALID_ARGUMENT , NOT_FOUND , PERMISSION_DENIED , UNAUTHENTICATED , RESOURCE_EXHAUSTED and DEADLINE_EXCEEDED .

- **API tests in CI, monitoring and observability** Manual exploration finds the bugs. Saved collections stop them from coming back. A mature team runs its API collection on every pull request and on a schedule against live environments.
- **Testing AI and LLM APIs and MCP servers** More and more products call large language model (LLM) APIs, or expose their own AI endpoints. These APIs behave differently from classic ones, and testers need new techniques.
- **Latest standards and trends** API standards keep moving. These are the ones worth knowing about today (checked in October 2026). Some are final standards, and some are drafts that tools already support.
- **An API test strategy and great bug reports** Always include the exact request (a curl command is ideal), the full response with status and body, the environment, the time in UTC and the request id. Remove real tokens and personal data before you paste.

TABLES

Table: reason · what it means in practice

Reason	What it means in practice
Earlier feedback	APIs are often ready before the screens. You can test the logic weeks before the UI exists.
Faster and more stable	A request takes milliseconds. No browser, no waiting for pages, no flaky locators.
Deeper coverage	The UI may block a negative quantity, but a direct API call can still send one. Attackers do not use your UI.
Many clients share one API	The web app, the Android app, the iOS app and partner systems all call the same API. One bug reaches every client.
Clear root cause	When a UI test fails you ask whether the page or the server broke. An API test points straight at the server.

Lesson: What an API is and why we test it

Table: type · what it confirms

Type	What it confirms
Functional	Each endpoint does what the contract says for valid and invalid input
Integration	Several APIs and the database work together in a full flow
Contract	The provider and its consumers still agree on the request and response shape
Security	Nobody can read or change data they should not
Performance	Response times stay within targets, also under load
Reliability	Timeouts, retries, duplicates and partial failures are handled safely
Usability of the API	Names, errors and documentation are clear for the developers who use it

Lesson: What an API is and why we test it

Table: point · api · web service

Point	API	Web service
Meaning	Any defined way for software to talk to other software	An API that is reached over a network with web protocols
Network	Not required. A Java library or an operating system call is also an API	Always needed
Protocols	Any: HTTP, gRPC, WebSocket, in process function calls	HTTP based: SOAP, REST, XML-RPC
Formats	Any: JSON, XML, binary Protocol Buffers, objects in memory	Mostly XML (SOAP) and JSON (REST)
Rule of thumb	**Every web service is an API, but not every API is a web service.**	

Lesson: What a web service is

Table: point · soap · rest

Point	SOAP	REST
What it is	A protocol with a fixed message format	An architectural style with guidelines
Message format	XML only, inside an Envelope with Header and Body	Any, usually JSON, also XML, plain text, files
Transport	Usually HTTP POST, can also use SMTP or JMS	HTTP, using GET, POST, PUT, PATCH and DELETE
Contract	WSDL, with XSD schemas for every message	OpenAPI (optional but common)
Errors	A `<soap:Fault>` element, usually with HTTP 500	HTTP status codes with an error body
Security	WS-Security standard (signatures, encryption inside the message)	HTTPS plus OAuth 2.0, JWT or API keys
State and transactions	Supports WS-AtomicTransaction and WS-ReliableMessaging	Stateless by design
Caching	Not cacheable, since everything is POST	GET responses can be cached
Typical users	Banks, insurance, telecom, government, older enterprise systems	Web and mobile apps, public APIs, microservices
Testing tools	SoapUI, ReadyAPI, Postman	Postman, curl, Bruno, Insomnia

Lesson: What a web service is

Table: element · what it describes · test idea

Element	What it describes	Test idea
`types`	XSD schemas for the data	Send values that break the schema (wrong type, missing required element)
`message`	The input and output messages	Check each output element is present and correctly typed
`portType` (`interface` in WSDL 2.0)	The operations, such as GetOrder	Call every operation, including ones the UI never uses
`binding`	Protocol and style (SOAP 1.1 or 1.2, document or rpc)	Use the right SOAPAction header and Content-Type
`service` and `port`	The endpoint address	Confirm QA points to the QA address, not production

Lesson: What a web service is

Table: constraint · meaning · what a tester checks

Constraint	Meaning	What a tester checks
Client and server	The UI and the data storage are separate and evolve independently	The API works the same when called from Postman, the app or another service
Stateless	Each request carries all the context, such as the token. The server keeps no session between calls	Send requests in a random order or from two tools at once. Results must not depend on a previous call
Cacheable	Responses state whether and how long they can be cached	`Cache-Control`, `ETag`, 304 responses, and no caching of private data
Uniform interface	Standard methods, URIs for resources, self descriptive messages, hypermedia	Correct methods and status codes, consistent naming, correct Content-Type
Layered system	Gateways, load balancers and caches can sit between client and server	Behaviour is the same through the gateway and directly. Headers are not lost on the way
Code on demand (optional)	The server may send executable code, such as JavaScript	Rare in APIs. If present, check it is served safely

Lesson: REST in depth

Table: type · who uses it · example · testing focus

Type	Who uses it	Example	Testing focus
Public or open	Any developer, often after signing up for a key	Weather, maps, GitHub API	Documentation accuracy, rate limits, abuse, backward compatibility
Partner	Selected business partners with an agreement	A payment gateway, a travel supplier	Contract terms, SLA, partner specific auth, onboarding sandbox
Internal or private	Teams inside the company	Order service called by the checkout service	Integration, contract tests between teams, zero trust auth
Composite	Clients that need several calls done as one	One call that creates an order, reserves stock and starts payment	Partial failure, rollback, performance

Lesson: Kinds of APIs and where they sit in a system

Table: component · job · what can go wrong

Component	Job	What can go wrong
Load balancer	Spreads requests across servers	One unhealthy server gives random failures
API gateway	Auth, rate limits, routing, request logging	Gateway rules differ between QA and production
Service	Business logic	Validation and rule bugs
Database	Stores data	Wrong data saved, slow queries, missing transactions
Cache (for example Redis)	Fast reads of popular data	Stale data after an update
Message broker (Kafka, RabbitMQ)	Passes events between services	Lost, duplicated or out of order events
Third party API	Payments, SMS, maps	Timeouts, changed responses, sandbox differences

Lesson: Kinds of APIs and where they sit in a system

Table: word · meaning

Word	Meaning
Endpoint	A method plus a URL that does one job, such as `POST /v1/orders`
Base URL	The common start of every URL, such as `https://api.shop.test/v1`
Payload	The data in the request or response body
Consumer and provider	The client that calls the API and the system that serves it
SDK	A ready made client library for an API in a given language
Sandbox	A safe test environment from a provider, with fake money or data
Throttling	Slowing down or refusing a client that sends too many requests
Serialization	Turning an object into JSON or XML. Deserialization is the reverse
Idempotent	Repeating the request leaves the same server state
Latency	Time taken for a request to get its response

Lesson: Kinds of APIs and where they sit in a system

Table: version · what changed · why a tester cares

Version	What changed	Why a tester cares
HTTP/1.1	Text based, one request at a time per connection, keep alive	Still the format you read in tools and logs
HTTP/2	Binary frames, many requests at once over one connection, header compression	Header names are always lowercase on the wire. gRPC runs on it
HTTP/3	Runs on QUIC over UDP, faster connection setup, better on weak mobile networks	Mostly invisible to API tests, but matters for performance on mobile

Lesson: How HTTP carries a request

Table: method · purpose · body · safe

Method	Purpose	Body	Safe	Idempotent	Typical success
`GET`	Read a resource or a list	No	Yes	Yes	200 OK
`POST`	Create a resource or trigger an action	Yes	No	No	201 Created, 200 or 202
`PUT`	Replace the whole resource	Yes	No	Yes	200 or 204
`PATCH`	Change some fields	Yes	No	Not guaranteed	200 or 204
`DELETE`	Remove a resource	Rarely	No	Yes	204 or 200
`HEAD`	Same as GET but headers only	No	Yes	Yes	200
`OPTIONS`	Ask which methods are allowed, used by CORS preflight	No	Yes	Yes	204 or 200

Lesson: HTTP methods, safety and idempotency

Table: code · name · when you should see it

Code	Name	When you should see it
200	OK	A successful GET, or an update that returns the resource
201	Created	A POST created something. A `Location` header should point to it
202	Accepted	Work was queued and will finish later, for example a report export
204	No Content	Success with an empty body, common for DELETE
301 / 308	Moved Permanently / Permanent Redirect	The resource has a new URL. 308 keeps the method and body
304	Not Modified	Your cached copy is still valid (after `If-None-Match`)
400	Bad Request	Malformed JSON, wrong type, missing required field
401	Unauthorized	No credentials, or invalid or expired credentials. It really means unauthenticated
403	Forbidden	You are known, but not allowed to do this
404	Not Found	The resource does not exist, or the API hides that it exists
405	Method Not Allowed	For example DELETE on a read only endpoint. An `Allow` header should list valid methods
409	Conflict	The request clashes with current state, such as a duplicate email
412	Precondition Failed	An `If-Match` check failed because someone else changed the resource
415	Unsupported Media Type	You sent XML to an endpoint that only accepts JSON
422	Unprocessable Content	Valid JSON, but the values break a business rule
429	Too Many Requests	You hit a rate limit. Look for `Retry-After`
500	Internal Server Error	Unhandled exception. Never acceptable as a response to bad input
502 / 503 / 504	Bad Gateway / Service Unavailable / Gateway Timeout	A proxy or gateway could not get a good answer from the service behind it

Lesson: Status codes

Table: header · direction · what to test

Header	Direction	What to test
`Content-Type`	Both	The format of the body. Send a wrong one and expect 415 or 400, not 500
`Accept`	Request	The format you want back. Ask for XML from a JSON only API and expect 406
`Authorization`	Request	Credentials such as `Bearer <token>`. Missing, malformed, expired and wrong user
`Cache-Control`, `ETag`	Response	Private data must not be cached by shared caches. ETag changes when the resource changes
`Location`	Response	Present on 201 and redirects, and the URL works
`X-Request-Id` or `traceparent`	Both	A correlation id you can give developers to find the exact log line
`Set-Cookie`	Response	Session cookies should have `HttpOnly`, `Secure` and a `SameSite` value
`Access-Control-Allow-Origin`	Response	CORS. A wildcard together with credentials is a security bug
`Strict-Transport-Security`	Response	Tells browsers to use HTTPS only
`Retry-After`	Response	With 429 or 503, how long to wait before retrying

Lesson: Headers, cookies and content negotiation

Table: style · how it works · typical tool · testing focus

Style	How it works	Typical tool	Testing focus
REST	Resources at URLs, HTTP methods as verbs, usually JSON	Postman, curl, Bruno	Methods, status codes, resource naming, pagination
SOAP	XML envelopes sent with POST, described by a WSDL file	SoapUI, Postman	XML schema (XSD) validation, SOAP faults, WS-Security
GraphQL	One endpoint. The client sends a query that names exactly the fields it wants	Postman, Insomnia, GraphiQL	Errors inside a 200 response, query depth, field level authorization
gRPC	Binary Protocol Buffers over HTTP/2, contract in a `.proto` file, supports streaming	grpcurl, Postman, Kreya	gRPC status codes, deadlines, streaming behaviour
WebSocket	A long lived two way connection for chat, live prices, games	Postman, wscat	Connect, message order, reconnects, auth on connect
Server-Sent Events	The server streams text events over one HTTP response	curl, Postman	Event format, streaming of AI responses, reconnect with Last-Event-ID
Webhooks	The server calls *your* URL when an event happens	webhook.site, ngrok, mock servers	Signature checks, retries, duplicate delivery

Lesson: REST, SOAP, GraphQL, gRPC, WebSocket and webhooks

Table: tool · best for · notes

Tool	Best for	Notes
Postman	Exploring, collections, environments, test scripts, collection runner	The most used tool in teams. Collections run in CI with the Postman CLI or Newman
curl	Quick checks from a terminal, sharing exact requests in bug reports	Installed on almost every machine. Every tool can import a curl command
Bruno	Offline, git friendly collections stored as plain text files	Popular since 2024 for teams that want collections reviewed in pull requests
Insomnia, Hoppscotch	Lightweight REST, GraphQL and WebSocket clients	Hoppscotch runs in the browser
Swagger UI, Redoc	Trying endpoints straight from the OpenAPI documentation	Good for a first look at a new API
SoapUI	SOAP and WSDL based services	Also does REST and simple load tests
Browser DevTools	Seeing the real calls a web page makes	Copy as cURL, throttle the network, block a request
Charles, Fiddler, mitmproxy	Intercepting mobile app traffic, editing requests in flight	Needed to test what a mobile app really sends

Lesson: Tools: Postman, curl, Bruno and friends

Table: tool · good at · notes

Tool	Good at	Notes
Charles Proxy	Mobile app testing on Mac, Windows and Linux	Breakpoints, rewrite rules, map local files, bandwidth throttling
Fiddler Everywhere and Fiddler Classic	Windows desktop and browser traffic	Auto responders that return a saved response for a matching request
Proxyman	A modern macOS and iOS friendly proxy	Easy certificate setup for iOS simulators
mitmproxy	Free, open source, scriptable in Python	`mitmweb` gives a browser interface
Burp Suite	Security testing: intercept, repeat, fuzz	Repeater and Intruder are covered in the security tools part

Lesson: Seeing real traffic: DevTools, HAR files and proxies

Table: line in the spec · test ideas

Line in the spec	Test ideas
`security: bearerAuth`	No token gives 401. Expired token gives 401. Another user's token cannot see this user's data
`required: [productId, quantity]`	Omit each one in turn. Send null. Send an empty body. Send `{}`
`type: integer`	Send `"2"`, `2.5`, `true`, `[]`, a very large number
`minimum: 1, maximum: 10`	Boundary values 0, 1, 2, 9, 10, 11 and a negative number
`maxLength: 200`	199, 200 and 201 characters, plus Unicode and HTML
`responses`	Can you trigger every documented status. Does the API ever return a status that is not documented

Lesson: Reading API docs and OpenAPI

Table: scheme · how it is sent · tester checklist

Scheme	How it is sent	Tester checklist
Basic auth	`Authorization: Basic base64(user:password)`	Only over HTTPS. Base64 is encoding, not encryption
API key	A header such as `x-api-key`, sometimes a query parameter	Keys in URLs leak into logs. Revoked keys must stop working
Bearer token	`Authorization: Bearer <token>`	Expiry, revocation on logout, token from another environment
JWT	A bearer token with three base64url parts: header, payload, signature	Change the payload and the signature must fail. `alg: none` must be rejected
OAuth 2.0	An authorization server issues access tokens with scopes	Scopes enforced, refresh flow, redirect URI exact match
OpenID Connect	OAuth 2.0 plus an ID token that says who the user is	ID token audience and expiry
Session cookie	`Cookie: session=...`	HttpOnly, Secure, SameSite, invalid after logout
mTLS	Both sides present certificates	Requests without the client certificate are refused

Lesson: Authentication and authorization

Table: type · the question it answers · example on a shop api · when it runs

Type	The question it answers	Example on a shop API	When it runs
Smoke	Is the build alive at all?	`GET /health` is 200, login works, one product loads	After every deploy, in minutes
Sanity	Does the area that just changed basically work?	The coupon fix: apply the coupon from the bug report once	After a small fix
Functional	Does each endpoint do what the spec says?	Create, read, update and delete an order with valid and invalid data	Every feature, every release
Negative	Does it fail safely and clearly?	Missing fields, wrong types, expired token, unknown id	Alongside functional tests
Integration	Do the services work together?	Placing an order really reduces stock and starts a payment	When services or contracts change
End to end (API level)	Does a full user journey work through the APIs?	Sign up, search, add to cart, pay, track the order	Before a release
Regression	Did the change break something that used to work?	The whole saved collection runs again	Every build in CI
Contract	Do provider and consumers still agree on the shape?	The mobile app still finds `price` as a number	On every provider change
Security	Can someone see or do what they should not?	Alice reads Bob's order by changing the id	Every release and after auth changes
Performance	Is it fast enough under real load?	p95 under 300 ms at 500 requests per second	Before big events and releases
Fuzz	What happens with random or weird input?	Thousands of generated bodies, looking for 500 errors	Nightly, or before a security review
Exploratory	What have we not thought of?	A 60 minute session on "refunds with two users and stale data"	When a feature is new or risky
Monitoring (synthetic)	Is production working right now?	A login and search check every 5 minutes from three regions	All the time, in production

Lesson: Types of API testing and when to use each

Table: field · value

Field	Value
ID and title	ORD-NEG-07: Quantity above the maximum is rejected
Preconditions	Logged in as a customer. Product 7 exists and is in stock
Request	`POST {{baseUrl}}/v1/orders` with body `{"productId":7,"quantity":11}`
Expected status	422
Expected body	Problem details with `errors[0].field = "quantity"` and a clear message
Expected side effects	No order row created. Stock unchanged
Priority and type	High, negative, boundary

Lesson: Designing API test cases

Table: category · values to try

Category	Values to try
Presence	Missing key, `null`, empty string, only spaces
Type	String for number, number for string, boolean, array, object
Size	Min length minus one, min, max, max plus one, 10,000 characters
Format	Invalid email, date in the wrong format, 30 February, negative zero
Characters	Unicode, emoji, right to left text, leading and trailing spaces, line breaks
Injection	`' OR 1=1 --`, `<script>`, `{{7*7}}`, `../../etc/passwd`
Duplicates	The same key twice in one JSON object, the same item twice in an array

Lesson: Designing API test cases

Table: style · request · strength · weakness

Style	Request	Strength	Weakness
Offset and limit	`?offset=40&limit=20` or `?page=3&size=20`	Jump to any page	Rows can repeat or be skipped when data changes between calls. Slow on large offsets
Cursor	`?after=eyJpZCI6NDB9&limit=20`	Stable and fast on large data	No jumping to page 7
Keyset	`?createdAfter=2026-10-01T00:00:00Z&limit=20`	Fast with an index	Needs a unique sort key

Lesson: Pagination, filtering and sorting

Table: bug · what happens · real world example

Bug	What happens	Real world example
Oversell	Two buyers get the last item	Flash sales, concert tickets, hotel rooms
Double submit	A double click or a retry creates two records	Two identical orders, a customer charged twice
Lost update	Two people edit the same record and the second save silently wipes the first	Two support agents editing one ticket
Limit bypass	A "use once" rule is broken by parallel requests	A one time coupon or a referral reward used five times at once
Balance check bypass	Two withdrawals both pass the "enough money?" check	A wallet that goes below zero

Lesson: Concurrency, race conditions and double clicks

Table: place · what to check · how testers look

Place	What to check	How testers look
Database	The row exists, every field has the right value and type, created and updated timestamps, no duplicate rows, nothing else changed	A read only SQL client (DBeaver, pgAdmin, MySQL Workbench), or a read endpoint of the API
Related data	Stock went down by exactly the quantity, loyalty points were added, the cart is now empty	Query the related tables or call the other services' read APIs
Message queue or event stream	One `OrderPlaced` event was published with the right ids, in the right format, only once	The Kafka, RabbitMQ or SQS console, or a test consumer
Notifications	Exactly one email or SMS, to the right person, in the right language, with the right amount	A test inbox such as Mailpit or MailHog in QA, or the provider's sandbox
Logs	No errors or stack traces for this request, no passwords, tokens or card numbers written in plain text	Search the log tool (Kibana, Splunk, Grafana Loki, CloudWatch) by the request id or trace id
Audit trail	Who did what and when, for sensitive actions such as refunds or role changes	The audit table or admin screen
Cache	After an update, the next read shows the new value, not an old cached one	Read twice, check `Age` and `Cache-Control` headers

Lesson: Checking side effects: database, queues, emails and logs

Table: test · expected

Test	Expected
Preflight from the real frontend origin	`Access-Control-Allow-Origin` echoes that origin, allowed methods and headers listed
Preflight from `Origin: https://evil.example`	No allow origin header for it, so the browser blocks the call
`Access-Control-Allow-Origin: *` together with `Access-Control-Allow-Credentials: true`	Never allowed. Browsers reject it, and an API that reflects any origin with credentials is a serious security bug
Origin `null` or a look alike such as `https://shop.com.evil.example`	Refused. Sloppy "starts with" or "contains" checks let these through

Lesson: HTTPS, TLS certificates, CORS and HTTP versions

Table: version · main idea · what testers notice

Version	Main idea	What testers notice
HTTP/1.1 (1997, now RFC 9112)	Text based, one request at a time per connection	Browsers open several connections. Headers are readable in raw form
HTTP/2 (RFC 9113)	Binary, many requests share one connection (multiplexing), headers compressed	gRPC needs it. Header names are lower case. DevTools shows h2 in the Protocol column
HTTP/3 (RFC 9114)	Runs over QUIC on UDP instead of TCP	Faster on lossy mobile networks and when switching from Wi-Fi to mobile data. Some corporate firewalls block UDP, so clients must fall back

Lesson: HTTPS, TLS certificates, CORS and HTTP versions

Table: risk · what it looks like · how to test by hand

Risk	What it looks like	How to test by hand
API1 Broken Object Level Authorization (BOLA)	Change `/orders/1042` to `/orders/1041` and see someone else's order	Use two users. Swap ids in every path, query and body
API2 Broken Authentication	Weak tokens, no lockout, tokens valid after logout	Brute force login a few times, replay old tokens, tamper JWTs
API3 Broken Object Property Level Authorization	Response shows `passwordHash`, or a PATCH accepts `"role":"admin"`	Read every field returned. Send extra fields in updates
API4 Unrestricted Resource Consumption	No limit on page size, file size or number of requests	`limit=100000`, huge uploads, many requests in a loop
API5 Broken Function Level Authorization	A customer can call DELETE `/admin/users/7`	Call admin and internal endpoints with a normal user token
API6 Unrestricted Access to Sensitive Business Flows	A bot buys all concert tickets or abuses referral credits	Repeat the flow quickly and at scale, check for limits and checks
API7 Server Side Request Forgery (SSRF)	An "import from URL" field fetches `http://169.254.169.254/`	Give internal addresses to any field that takes a URL
API8 Security Misconfiguration	CORS allows any origin, debug mode on, missing security headers	Check headers, error pages, OPTIONS responses, TLS settings
API9 Improper Inventory Management	An old `/v1` still runs without the fixes that `/v2` has	Try older versions and staging hosts. Compare behaviour
API10 Unsafe Consumption of APIs	The API trusts data from a partner API without validation	Mock the third party with bad or malicious responses

Lesson: API security testing: the OWASP API Top 10

Table: term · meaning

Term	Meaning
Latency or response time	Time from sending the request to receiving the full response
Throughput	Requests handled per second (RPS)
Percentile (p95, p99)	p95 = 300 ms means 95% of requests took 300 ms or less
Error rate	Share of requests that failed, often with a target such as below 0.1%
Concurrency	Requests in progress at the same time

Lesson: Performance testing for APIs

Table: risk · test

Risk	Test
Price tampering	Send `"price": 1` in the cart body. The server must use its own price
Stock race	Two users buy the last item at the same moment. Only one order succeeds
Double charge	Retry the payment call with the same Idempotency-Key. Only one charge
Coupon abuse	Apply the same single use coupon in two parallel requests
Payment webhook lost	The order must not stay "pending" forever. A reconciliation job should fix it

Lesson: Real world scenario walkthroughs

Table: risk · test

Risk	Test
Negative amount	`"amount": -500` must be rejected, or money flows backwards
Precision	0.1 plus 0.2 and amounts with three decimals. Money must be exact
Atomicity	Debit succeeds and credit fails. Both must roll back
Limits	Daily limit exactly reached, then one rupee more
OTP	OTP brute force limit, OTP reuse, OTP from another transaction

Lesson: Real world scenario walkthroughs

Table: risk · test

Risk	Test
Location data	Latitude 91, longitude 181, null island (0, 0), and spoofed jumps across the city
Real time updates	WebSocket drops and reconnects. Driver position must resume without duplicates
State machine	Cancel after the trip ended. Rate a trip that never happened
Fare	Surge multiplier applied once, not twice. Fare shown equals fare charged
Privacy	Rider phone number is masked in the driver API response

Lesson: Real world scenario walkthroughs

Table: strategy · example

Strategy	Example
URL path	`/v1/orders` and `/v2/orders`
Header	`Api-Version: 2026-10-01` (date based, used by Stripe and others)
Media type	`Accept: application/vnd.shop.v2+json`

Lesson: Contract testing and versioning

Table: tool · use

Tool	Use
Postman mock servers	Return saved examples for a collection. Quick for front end teams
WireMock	Rich request matching, fault injection, delays, stateful scenarios
Prism	Starts a mock straight from an OpenAPI file and validates requests against it
Mockoon, MockServer, Microcks	Desktop and container mocks, Microcks also handles async APIs

Lesson: Mocks, service virtualization and test data

Table: tool · what it is · how testers use it

Tool	What it is	How testers use it
OWASP ZAP (Zed Attack Proxy)	A free, open source security scanner and proxy, now maintained by Checkmarx under the name ZAP	Import an OpenAPI file, run the API scan, review alerts such as missing security headers, injection and information leaks. Runs in CI with a Docker image
Burp Suite (Community and Professional)	An intercepting proxy built for security testing	**Repeater** to edit and resend one request many times, **Intruder** to fuzz a parameter with a word list, **Comparer** to diff two responses, extensions such as Autorize to find authorization bugs automatically
Schemathesis	An open source tool that reads your OpenAPI or GraphQL schema and generates thousands of test requests from it (property based testing)	`schemathesis run https://api.shop.test/openapi.json` finds 500 errors, responses that break the schema and requests the API wrongly accepts
RESTler	Microsoft Research's stateful REST fuzzer	Learns which calls produce ids for other calls, then fuzzes whole sequences
Postman and Newman with data files	Not a fuzzer, but a CSV of nasty values works well	Run one request with hundreds of rows of bad input

Lesson: Security tools and fuzzing: ZAP, Burp Suite and Schemathesis

Table: difference · how to test it

Difference	How to test it
Non deterministic output	The same prompt can give different words. Assert on structure, required facts and rules, not on exact text. Run the same case several times
Streaming responses	Answers arrive as Server-Sent Events. Check the event order, the final event, and what happens if the stream is cut
Structured output	When the API is asked for JSON, validate it against a schema every time
Token limits and cost	Very long inputs, the max tokens setting, truncated answers, and usage numbers in the response
Tool calling	The model asks your API to run a function. Check arguments are validated like any other untrusted input
Safety	Prompt injection hidden in user content or documents, data leaks across users, harmful requests refused
Rate limits	Limits in requests and tokens per minute, 429 and 529 style overload responses, retry with backoff

Lesson: Testing AI and LLM APIs and MCP servers

Table: topic · status · what it means for testers

Topic	Status	What it means for testers
RFC 9457 Problem Details	Standard (2023)	One error format across APIs. Check `application/problem+json`
OpenAPI 3.1 and 3.2	3.2 released September 2025	Full JSON Schema, QUERY method, streaming responses, better tags
Arazzo 1.0	OpenAPI Initiative standard (2024)	Describes multi step API workflows, such as login then order then pay. Useful as a source for end to end tests
AsyncAPI 3.0	Standard (2023)	The OpenAPI equivalent for Kafka, MQTT and WebSocket based APIs
HTTP QUERY method	RFC 10008 (June 2026)	A safe, idempotent request that carries a body, for complex searches that do not fit in a URL. Rest Assured 6.1 and newer can already send it. Test that QUERY never changes data and that caches and gateways pass it through
RateLimit headers	IETF draft	Standard `RateLimit` and `RateLimit-Policy` headers to replace the many `X-RateLimit` variants
Idempotency-Key header	IETF draft	Standardizes the header payment APIs already use
Deprecation header	RFC 9745 (2025)	A standard way to announce deprecated endpoints
OAuth 2.1	IETF draft, still in progress in 2026	PKCE for every client, no implicit or password flow, exact redirect matching
DPoP	RFC 9449 (2023)	Tokens bound to a key the client holds, so a stolen token alone is useless
HTTP/3	RFC 9114 (2022)	Faster on mobile networks. Check your tools and gateways support it
MCP	Open protocol, latest specification dated 2026-07-28	AI agents calling tools. The 2026 revision made the core stateless, added `Mcp-Method` and `Mcp-Name` routing headers and moved tool schemas to JSON Schema 2020-12
OWASP Top 10:2025	Published for web apps (the API Top 10 is still the 2023 edition)	Adds software supply chain failures and mishandling of exceptional conditions. Both affect APIs: risky dependencies and error paths that fail open

Lesson: Latest standards and trends

Table: release · what is new · how it helps a tester

Release	What is new	How it helps a tester
Postman 12	Git native workspace, Collection 3.0 in YAML, code based local mock servers, Agent Mode, API Catalog	Collections live in the same pull request as the code change, so reviewers see the API change and its tests together. Local mocks let you test before the backend is ready
Rest Assured 6.x (6.3.0, October 2026)	QUERY method, JSON Schema drafts up to 2020-12, mutual TLS client certificates, clearer error messages	Validate modern OpenAPI 3.1 schemas and test banking style mTLS APIs from the same framework
pytest 9	Built in subtests, native TOML configuration, a strict mode	One test can check many fields and report every failure instead of stopping at the first
Playwright 1.6x	Tests for WebMCP tools, passkey (WebAuthn) support, isolated retries, test locks	UI and API tests for AI features and passwordless login in one runner
Schemathesis 4	A rewritten core engine, plus stateful testing that follows OpenAPI links from one operation to the next	Point it at an OpenAPI file in CI and it finds 500 errors and schema breaks nobody wrote a test for
OpenAPI 3.2 and Arazzo 1.0	Streaming responses, the QUERY method, multi step workflow descriptions	Generate end to end flows such as "log in, add to cart, pay" from a reviewed file instead of from memory

Lesson: Latest standards and trends

TIPS AND TRAPS

REMEMBER

In the test pyramid, API tests sit in the middle layer, also called the service or integration layer. Teams usually want many unit tests, a solid set of API tests and only a few end to end UI tests, because the cost and the run time grow as you go up.

Lesson: What an API is and why we test it

REMEMBER

In SoapUI or Postman, import the WSDL and the tool creates a sample request for every operation. SOAP 1.1 uses `Content-Type: text/xml` plus a `SOAPAction` header. SOAP 1.2 uses `application/soap+xml`.

Lesson: What a web service is

IN THE EXAM

"Is REST a protocol?" is a favourite interview trap. No. REST is an architectural style. HTTP is the protocol most REST APIs use. SOAP is a protocol.

Lesson: What a web service is

REMEMBER

The meaning of methods, status codes and headers is the same in all three versions. It is defined in one document, RFC 9110 (HTTP Semantics), published in 2022.

Lesson: How HTTP carries a request

COMMON MISTAKE

A GET that changes data, such as `GET /orders/5/cancel`, is a design bug. Browsers, crawlers and caches may call GET links on their own and cancel real orders. Report it.

Lesson: HTTP methods, safety and idempotency

IN THE EXAM

Idempotent does not mean the same response. The first `DELETE /orders/5` returns 204 and the second returns 404. The **server state** is the same after both (order 5 is gone), so DELETE is still idempotent.

Lesson: HTTP methods, safety and idempotency

REMEMBER

A 5xx caused by your input is always a bug. Bad input should give a 4xx with a helpful message. When you see a 500 with a stack trace in the body, report two bugs: the missing validation and the information leak.

Lesson: Status codes

REMEMBER

In browser DevTools, open the Network tab, click any request and look at the Headers panel. Right click the request and choose Copy as cURL to replay it in a terminal or import it into Postman. This is the fastest way to start testing an API behind a web page.

Lesson: Headers, cookies and content negotiation

REMEMBER

Never save real passwords or production tokens in a shared collection. Use the environment's secret variable type or a vault, and keep the current value local so it is not synced.

Lesson: Tools: Postman, curl, Bruno and friends

REMEMBER

A quick win: compare what the app sends with the API documentation. Extra fields nobody documented, personal data sent to analytics, old API versions still being called and tokens passed in URLs are all common findings.

Lesson: Seeing real traffic: DevTools, HAR files and proxies

COMMON MISTAKE

A HAR file can contain passwords, cookies, tokens and personal data. Chrome now strips cookies and Authorization headers from exported HAR files by default, but other tools may not. Before you attach a HAR to a ticket, open it in a text editor or a sanitizer tool and remove secrets.

Lesson: Seeing real traffic: DevTools, HAR files and proxies

REMEMBER

Also test what the spec does not say. Send unknown extra fields and see whether they are ignored, rejected with 400, or saved. If they are saved, you may have found the mass assignment bug covered in the security part.

Lesson: Reading API docs and OpenAPI

REMEMBER

In real teams, the same request can serve several test types. A single "create order" call can sit in the smoke suite, the regression suite and the performance script. What changes is the question you ask and how often you run it.

Lesson: Types of API testing and when to use each

COMMON MISTAKE

Watch for responses that wrap errors inside a 200, such as `{"success": false, "error": "Invalid id"}`. Clients and monitors that only check the status will treat failures as success. Report it as a design issue.

Lesson: What to check in every response

REMEMBER

Every test must create its own data or clean up after itself. A test that depends on an order created by yesterday's run will fail the day someone resets the database.

Lesson: Variables, chaining and end to end flows

IN THE EXAM

Offset pagination with a changing data set is a classic interview question. If a new item is inserted at the top while you read page 1, the last item of page 1 moves to page 2 and you see it twice. Cursor pagination avoids this.

Lesson: Pagination, filtering and sorting

REMEMBER

Test the same key with a *different* body. A well designed API returns an error such as 422 or 409, because reusing a key for a different payment is a client bug, not a retry.

Lesson: Rate limits, caching and idempotency keys

REMEMBER

Very large uploads on poor mobile networks often use **resumable uploads**, where the file is sent in chunks and an interrupted upload continues where it stopped. The open tus protocol and the cloud providers' resumable APIs work this way. Test by switching the network off halfway and back on.

Lesson: File uploads and downloads

REMEMBER

Every API that accepts a date, an amount or a name should have at least one test with a time zone other than the server's, one with an awkward amount, and one with non English text.

Lesson: Dates, money, text and language edge cases

COMMON MISTAKE

After a race test, always check the data, not just the status codes. Two 201 responses might be correct (two different items), while one 201 and one 500 might still have left half finished data behind.

Lesson: Concurrency, race conditions and double clicks

REMEMBER

If you have no database access, ask for it in read only form, or ask developers for "test support" endpoints in QA that show internal state. Being able to see side effects is one of the biggest differences between a beginner and a senior API tester.

Lesson: Checking side effects: database, queues, emails and logs

REMEMBER

CORS is enforced only by browsers. Postman and curl ignore it completely, so "it works in Postman but not in the app" is often a CORS problem. To test CORS by hand, add an `Origin` header in curl and read the response headers: `curl -i -X OPTIONS https://api.shop.test/v1/orders -H "Origin: https://shop.test" -H "Access-Control-Request-Method: PUT"`

Lesson: HTTPS, TLS certificates, CORS and HTTP versions

COMMON MISTAKE

Security testing on systems you do not own or have no written permission to test can be illegal. Use your team's QA environment, or practice on deliberately vulnerable apps such as OWASP crAPI and OWASP Juice Shop.

Lesson: API security testing: the OWASP API Top 10

COMMON MISTAKE Scanners and fuzzers send a lot of traffic and can create, change or delete data. Run them only against environments you own or have written permission to test, never against production unless the team has planned it, and tell the team before a big run.

Lesson: Security tools and fuzzing: ZAP, Burp Suite and Schemathesis